

Method and Experiments

3. Method

3.1 Setting

We study in-context policy improvement with a single softmax attention layer of fixed weights. The agent acts in a tabular MDP; each state–action pair is embedded by a fixed linear feature map $\phi(s,a) \in \mathbb{R}^d$, and the action-value function is linear, $Q(s,a) = \phi(s,a)^T w$. At each update step the layer receives a trajectory of transitions (s, a, r, s') and produces a parameter update dw that improves the policy. As a reference upper bound we use the exact semi-gradient Q-learning update, $dw = (\alpha/n) \sum_t \delta_t \phi_t$, implemented in numpy.

3.2 Softmax evaluation with a linear readout improvement

A softmax block cannot carry an unnormalised bilinear term in its scores, so the improvement residual must be assembled from quantities the layer can represent. We therefore separate evaluation from improvement. The prompt matrix $Z \in \mathbb{R}^{D \times (n+1)}$, with $D = d+3$, stacks per time step the features $\phi(S_t, A_t)$ (rows 0..d-1), the reward R_{t+1} (row d), a target memory (row d+1, implemented as a temporal-shift memory column), and a current-value memory $v(S_t, A_t)$ (row d+2). The layer update is

$$Z_{\text{half}} = Z + V Z \text{softmax}(Z^T A Z + M),$$

where the mask M forbids the query column from attending to itself as a source, and the target memory is shifted as $\text{target} \leftarrow \gamma \cdot \text{current}$. The evaluation output is the updated current-memory row, and the improvement is the semi-gradient readout

$$\delta_t = R_{t+1} + \gamma v_{t+1} - v_t, dw = (\alpha)/(n) \sum_{t=1}^n \delta_t \phi(S_t, A_t),$$

with the closed loop $w \leftarrow w + dw$ and $v(\cdot) \leftarrow v_{\text{pred}}(\cdot)$ across frames. Two design constraints make this realisable by a softmax layer: w never enters the scores, and evaluation is carried by the memory rows, which are independent of w . The model thus contains no bilinear term and no w -dependent scores; it is a single layer with a linear V and a fixed kernel A initialised as a feature inner product (model.py, SoftmaxEvalImproveV2).

3.3 Score-scaled softmax operator in the attention weights

The convex-hull constraint can be addressed directly by moving the improvement operator inside the attention weights rather than the readout. The TD errors are multiplied back into the weights,

$$M = \max_{a'} \phi(S', a')^T w, \delta_j = R_j + \gamma M_j - \phi(S_j, A_j)^T w,$$

$$dw = \sum_j \delta_j \cdot \text{softmax}(\delta_j / \tau)_j \cdot \phi(S_j, A_j).$$

As $\tau \rightarrow \infty$, $\text{softmax}(\delta/\tau)$ approaches the uniform vector $1/n$ and the update degenerates exactly to the semi-gradient Q-learning update $(1/n) \sum_j \delta_j \phi_j$. To isolate whether competitive normalisation is the root cause of the constraint, we also compare per-token signed gates, $\tanh(\delta_j/\tau)$ and $(2\sigma(\delta_j/\tau)-1)$, with a τ -scaled step-size compensation so that the large- τ step does not vanish. The mechanism is tested in numpy (diag_attlin.py); the softmax is numerically stabilised by subtracting the maximum before exponentiation.

4. Experiments

4.1 Setup

Random tabular MDPs. We use $nS=9$ states, $nA=4$ actions, and discount $\gamma=0.5$, with randomly drawn transition probabilities and rewards (sample_mdp); each state–action pair is embedded by a random Gaussian feature vector in $\mathbb{R}^{36}$ ($d = nS \cdot nA$, sample_features). Training runs 400 update steps per MDP over 100 MDPs and 3 seeds (independent MDPs, features, and initial w per seed, sharing the same trained checkpoint), with exploration $\epsilon=0.1$, step size $\alpha=0.2$, and trajectory length $n=20$. Greedy policies are evaluated by Monte-Carlo return over 32 trajectories of length 50. Baselines are the oracle (value-iteration greedy policy), a random policy, one-step greedy behaviour (0.376), and the exact numpy semi-gradient Q-learning update as the upper bound (0.428–0.461).

Deterministic grid worlds. For generalisation we use deterministic 6×6 grid worlds with a goal and a pit (absorbing, reward $+1 / -1$, step penalty -0.04 , $\gamma=0.9$) and spatial RBF features $\phi(s,a)=\psi(s) \otimes e_a$ with a 3×3 Gaussian centre grid ($d=36$, `rbf_features`); 40–60 MDPs with random goal/pit placements. Training runs 2500 update steps with exploration $\epsilon=0.3$, and every rollout restarts from the start state (episodic reset, see Section 4.5). Unless stated otherwise, reported returns are final Monte-Carlo returns over 32 trajectories of length 60.

4.2 Main result: softmax evaluation with a linear readout

On random tabular MDPs, a single softmax layer with a linear readout achieves closed-loop returns of 0.42–0.45, matching the numpy semi-gradient teacher and exceeding one-step greedy behaviour (0.376), while approaching the oracle (0.53). Table 1 reports the per-seed results across hard and softmax-max variants of the greedy bootstrap. Returns improve monotonically with context length, from 0.38 at 200 frames to 0.43 at 400 frames. These results provide, to our knowledge, the first constructive demonstration that softmax attention can perform Q-learning-style policy improvement; the mechanism tracks the semi-gradient upper bound rather than one-step greediness.

Table 1. Closed-loop returns of the C3-v3 layer (3 seeds $\times$ max type).

seed	max type	C3-v3	numpy qlearn	oracle
0	hard max	0.4266	0.4422	0.5283
0	softmax $\tau=0.3$	max, 0.4206	0.4519	0.5283
1	hard max	0.4395	0.4653	0.5354
1	softmax $\tau=0.3$	max, 0.4321	0.4661	0.5354
2	hard max	0.4282	0.4070	0.5325
2	softmax $\tau=0.3$	max, 0.4420	0.4279	0.5325
mean		0.4315	0.4434	0.5323

4.3 Score-scaled operator and gate variants

For $\tau \geq 5$, the score-scaled operator (`att td lin`) reaches closed-loop returns of 0.43–0.48, matching or exceeding numpy qlearn (0.428–0.461); the tanh and sigmoid gates also close the loop, at 0.40–0.48 and 0.39–0.45 respectively. The improvement operator can therefore live inside the attention weights: the sign of the update is recovered from the multiplied-back δ , and in the large- τ limit the operator is exactly semi-gradient learning. A finer temperature scan (Section 4.7) shows a mild optimal window $\tau \approx 5$ –20 in the dense regime rather than monotone improvement in τ .

4.4 Boundary: self-containment of the current value

The model cannot maintain $\phi^T w$ on its own. If w is excluded from the input, the model degenerates to a copy operator and the evaluation drifts; if w is included, a single softmax layer reconstructs $\phi^T w$ only partially, at a structural floor of $R^2 \approx 0.92$, because softmax normalisation discards the unnormalised $\Phi^T \Phi w$ term that linear attention can carry. (The R^2 figure normalises the residual MSE by the variance of $\phi^T w$ for freshly drawn w ; the residual floor itself is protocol-independent — in our reproduction the evaluation loss plateaus at 0.15–0.27 regardless of training length or depth — and it is this non-vanishing residual, not the exact R^2 value, that breaks the closed loop.) Improvement is therefore realisable, whereas self-containment of the value is not; this is a structural boundary of softmax improvement rather than a training artefact.

4.5 Generalisation: deterministic grid worlds

Mechanism transfer. On dense-reward deterministic grid worlds (Section 4.1), every signed mechanism transfers: `qlearn`, `att td lin` ($\tau \geq 5$), and the tanh gate all converge to the RBF linear ceiling of ≈ 5.0 –5.5, far above random (≈ 3.2) and approaching the oracle (≈ 7.3); the gap to the oracle is explained by the feature ceiling (the 3×3 RBF centre grid cannot represent $Q^{\{*\}}$ exactly), not by the mechanism. The low-temperature regime reproduces across settings: `att td lin` at $\tau=1$ lags clearly (4.05 vs 5.2). Under sparse rewards with episodic reset, the same mechanisms reach 0.29–0.38, well above random (-0.31).

Episodic reset is a protocol requirement, not a mechanism property. An earlier protocol

continued the trajectory across rollouts (resuming from the last state); under it, every mechanism, including tabular Q-learning run for 500k steps, collapsed to random — a state-distribution starvation: the start state was sampled essentially once, and once the behaviour locked into the goal basin, the start state's values were never updated while the Monte-Carlo evaluation kept starting there. Restarting every rollout from the start state (episodic reset) fixes this: dense tabular Q-learning climbs from 2.5 to the oracle (7.6). Random MDPs avoid the pathology only because their start distribution is dense and rewards are dense everywhere. This is a protocol constraint on any TD learner in the closed loop, not a failure of the softmax mechanism.

4.6 Real single-layer softmax block on grid worlds

The grid-world results above simulate the mechanisms with hand-written linear algebra. We verify with the real softmax block (`SoftmaxEvalImproveV2`): a single layer trained to imitate the teacher readout $dw = (\alpha/n)\Phi^T \delta$ on grid worlds (400 MDPs $\times$ 30 steps, loss $\rightarrow 0$) and then evaluated closed-loop on 60 new MDPs with an externally supplied live $\phi^T w$. Across 3 seeds the model reaches ≈ 5.31 , matching the teacher ≈ 5.26 and the RBF ceiling ≈ 5.08 , far above random ≈ 3.14 — the C3-v3 conclusion transfers to structured MDPs. With self-evaluation (the layer maintains its own value memory, no external $\phi^T w$) the same model does not improve at all and ends below random (2.40–2.75 vs random 3.06–3.21 across seeds). The self-containment boundary is a universal obstacle, not an artefact of the random-MDP setup.

4.7 Temperature phase transition

A continuous scan of τ (13 values from 0.1 to 50) on dense grid worlds confirms a smooth phase transition rather than a binary regime. The critical temperature is $\tau_c \approx 3$ –5: for $\tau \in [0.1, 2]$ returns sit at 3.4–4.3 (with a valley near $\tau=0.5$), from $\tau=3$ they climb, and for $\tau \geq 10$ they saturate at the ceiling (≈ 5.2 –5.5). Finite τ shows a mild regularisation gain: $\tau=10$ reaches 5.49, slightly above the pure semi-gradient qlearn (5.42), consistent with the second-order term of the first-order expansion weighting large-error samples.

Under sparse rewards the phase transition flips: the optimal temperature moves to the low end. $\tau=1$ –2 reaches 0.42–0.46, exceeding qlearn (0.34) by up to 36%, while $\tau=0.3$ dies (–0.26) and high temperatures fall back (0.29 at $\tau=50$). The temperature acts as a signal–noise matching filter: with sparse signals concentrated on the few transitions near the goal, a low temperature makes the softmax select the single strongest- δ token each step and focus learning on the informative transitions, whereas a high temperature averages the sparse signal into TD noise. On random MDPs with dense rewards the optimum stays at $\tau \approx 10$. The optimal temperature therefore flips with reward sparsity, and τ is a tunable hyper-parameter rather than a parameter to take to the large limit.

4.8 Mechanism attribution: smoothness $\times$ geometric alignment

Which ingredient makes low temperature a signal extractor rather than a variance trap? We decompose the variables with controlled comparisons on random tabular MDPs and grid worlds. Sparse rewards alone, determinism alone, and goal-shaped rewards alone each fail to revive $\tau=1$ on random MDPs. The decisive variable is the feature geometry. On the same deterministic sparse grid world, replacing the RBF features by random features flips $\tau=1$ from working (0.38) to collapsing below random (–0.38). Randomising the position mapping used by the RBF kernel (so the features remain smooth but no longer aligned with the grid adjacency) flips $\tau=1$ from exceeding qlearn on every seed (0.33–0.45) to 0.07–0.21 (3 seeds), far below the aligned regime, and simultaneously collapses the qlearn expression ceiling (0.06–0.13 vs 0.27–0.46 aligned). An artificially smooth but unaligned RBF on random MDPs does not revive low temperature (0.29 < qlearn 0.38). Low-temperature one-hot focusing is therefore useful exactly when the features are both smooth and geometrically aligned with the transition structure: $Q = \psi(s)^T w$ is then spatially smooth, the TD error field has spatial structure concentrated near the goal, and the strongest- δ token is stable.

Direct probes of the δ field confirm this mechanism. Aligned RBF features produce a spatially smooth steady-state δ field (adjacent-state max- δ differences 0.64–0.69, vs 1.02–1.10 for permuted and 0.92–1.06 for random features). During early training, when $Q \approx 0$ and $\delta \approx r$, the δ field is strongly concentrated near the goal (Spearman $\rho \approx +0.75$ –0.79 between max- δ and inverse distance-to-goal at steps 50–250), then drifts to $\rho \approx +0.1$ –0.3 once Q is learned and δ becomes a Bellman residual; with random features ρ is negative throughout (–0.15 to –0.55), so the one-hot focus is misdirected and $\tau=1$ collapses. Geometric alignment simultaneously guarantees the early focus (Q is smoothly learnable) and the steady-state smoothness of the δ field.

5. Discussion

Mechanism. A pure convex-combination update (softmax weights non-negative and summing to

one) loses the sign of δ and deadlocks; multiplying δ back into the weights restores the sign, and in the $\tau \rightarrow \infty$ limit the operator reduces exactly to semi-gradient learning. The boundedness of closed-loop gains is consistent with the two structural obstacles identified in Section 2 — the convex hull [13] and the degradation of softmax sharpness with sequence length [14]. The temperature is not a monotone dial: it acts as a signal-noise matching filter whose optimum flips with reward sparsity (Section 4.7), and low-temperature focusing is useful only under smooth, geometrically aligned features (Section 4.8). The one structurally fatal operator remains the unscaled pure convex combination.

Connection to prior work. The paper fills the gap between linear-attention policy improvement [4] and softmax-attention policy evaluation [5]. The score-scaled operator is the softmax-side analogue of the semi-gradient SARSA / Q-learning update that linear attention realises exactly [4]; its large- τ limit is exactly that update. The self-containment floor is the closest softmax-side analogue to covariance-readout analyses of attention outputs [15] and to the weight-learning results established for linear attention [16].

Limits and future work. We have not established a theoretical convergence bound; the evidence is experimental. Closed-loop improvement is demonstrated on dense-reward random tabular MDPs and on deterministic grid worlds with smooth, aligned features; the low-temperature regime requires such aligned features to be useful (Section 4.8). The self-containment boundary ($R^2 \approx 0.92$) is measured, not proved; a proof would require the bilinear reconstruction analysis we leave open. A tighter theoretical account of the phase-transition critical temperature τ_c and of the alignment conditions would also be valuable.

6. Reproduction commands

```
# C3-v3 closed-loop main result (random tabular MDPs)
python evaluate_c3_v3.py --ckpt results/out_c3_v3/model.pt --self-eval 0 --update-steps 400
# Score-scaled operator / gates (random tabular MDPs)
python diag_attlin.py
# Grid-world generalisation (dense/sparse, must use --reset 1)
python diag_gridworld.py --feat rbf --dense --reset 1 --eps 0.3 --update-steps 2500 --n-mdps 40
python diag_gridworld.py --feat rbf --reset 1 --eps 0.3 --update-steps 2500 --n-mdps 40
# Real softmax block on grid worlds (external  $\phi^*w$  vs self-evaluation)
python train_c3_v3_gridworld.py --dense --outdir results/out_c3_v3_gw
python evaluate_c3_v3_gridworld.py --ckpt results/out_c3_v3_gw/model.pt --dense --self-eval 0
python evaluate_c3_v3_gridworld.py --ckpt results/out_c3_v3_gw/model.pt --dense --self-eval 1
# Temperature phase transition (dense/sparse)
python diag_gridworld_tauscan.py --dense --eps 0.3 --update-steps 2500 --n-mdps 40 --seed 0
python diag_gridworld_tauscan.py --eps 0.3 --update-steps 2500 --n-mdps 40 --seed 0
# Alignment attribution + delta-field probes
python diag_gridworld_featalign.py --eps 0.3 --update-steps 2500 --n-mdps 40 --seed 0
python diag_gridworld_deltaprobe.py --n-mdps 15 --seed 0
python diag_gridworld_deltaevolve.py --seed 0
```